# SRM Institute of Science and Technology

**(Deemed to be University U/S 3 of UGC Act, 1956)**

## FACULTY OF SCIENCE AND HUMANITIES

**Tiruchirapalli, Tamil Nadu-621105.**

---

**Department of Computer Applications**

PRACTICAL RECORD

**NAME** :

**REGISTER NO** :

**COURSE** : **MCA - GENERATIVE AI**

**SEMESTER / YEAR** : **III / II**

**SUBJECT CODE** : **PGI25D09J**

**SUBJECT NAME** : **IoT Devices with Computer Vision Technologies**

**November 2026**

**SRM Institute of Science and Technology**

**(Deemed to be University U/S 3 of UGC Act, 1956)**

**FACULTY OF SCIENCE AND HUMANITIES**

**Tiruchirapalli, Tamil Nadu-621105.**

---

**Department of Computer Applications**

# BONAFIDE CERTIFICATE

This is to certify that the bonafide work is done by Mr/Ms _____

Register No. _____ in **IoT Devices with Computer Vision Technologies (Subject Code: PGI25D09J)** at Computer Lab, SRM UNIVERSITY, Tiruchirapalli Campus, in October 2026.

**STAFF IN CHARGE** **HEAD OF THE DEPARTMENT**

Submitted for the University Practical Examination held at SRM University, Tiruchirapalli Department of Computer Applications on _____

**INTERNAL EXAMINER** **EXTERNAL EXAMINER**

IoT Devices with Computer Vision Technologies

Code : PGI25D09J Class: II MCA-Gen AI


| S.NO | DATE | NAME OF THE PROGRAMS | PAGE.NO | SIGNATURE |
|---|---|---|---|---|
| 1 | 16.07.2026 | Explain working of Raspberry Pi | 1 | |
| 2 | 16.07.2026 | Controlling LED with Raspberry Pi | 4 | |
| 3 | 23.07.2026 | Interfacing Light Sensor with Raspberry Pi | 6 | |
| 4 | 23.07.2026 | Describe gateway as a service deployment in IoT toolkit | 8 | |
| 5 | 30.07.2026 | Weather Monitoring System | 10 | |
| 6 | 30.07.2026 | IoT based Soil Moisture Monitoring Device | 12 | |
| 7 | 06.08.2026 | Install OpenCV, Displaying images OpenCV | 14 | |
| 8 | 20.08.2026 | Reading & Writing images OpenCV | 16 | |
| 9 | 20.08.2026 | Draw a Rectangle, Draw a Circle | 18 | |
| 10 | 27.08.2026 | Text in Images | 20 | |
| 11 | 03.09.2026 | Color Space OpenCV Thresholding OpenCV | 22 | |
| 12 | 10.09.2026 | Finding Contours | 24 | |
| 13 | 17.09.2026 | Image Edge Detection OpenCV | 26 | |
| 14 | 24.09.2026 | Image Scaling & Rotation using OpenCV | 28 | |
| 15 | 01.10.2026 | Image Translation OpenCV, Image Filtering OpenCV | 30 | |

# INTERNET OF THINGS LAB
# PRACTICAL LAB OBSERVATION

## Ex. No: 1 EXPLAINING THE WORKING OF RASPBERRY PI

**Date:16.07.2026**

**AIM:**

To understand the working of the Raspberry Pi (its hardware blocks, boot process and GPIO interface) and to display the board and system details using a Python program.

**PROCEDURE:**

Step 1: Study the main blocks of the Raspberry Pi: ARM processor (SoC), RAM, GPIO header, USB, HDMI, Ethernet/Wi-Fi and the microSD card slot.

Step 2: Write the Raspberry Pi OS image to a microSD card and insert it into the board. Connect the keyboard, monitor and 5V power supply.

Step 3: Power on the board and observe the boot sequence: GPU firmware loads, then the kernel, then the desktop or terminal.

Step 4: Open the terminal and create a Python program that reads the board model, OS, CPU temperature and memory details from the system files.

Step 5: Run the program and note the hardware and software details displayed.

**CODE:**

```
import os, platform, socket

def read(path):
    with open(path) as f:
        return f.read().strip('\x00\n ')

model = read('/proc/device-tree/model')
temp = int(read('/sys/class/thermal/thermal_zone0/temp')) / 1000

mem = {}
with open('/proc/meminfo') as f:
    for line in f:
        k, v = line.split(':')
        mem[k] = int(v.split()[0]) / 1024 / 1024   # kB -> GB

print("Hostname      :", socket.gethostname())
print("Board Model  :", model)
print("Operating Sys :", platform.system(), platform.release(), f"({platform.machine()})")
print("Python Version:", platform.python_version())
```

```
print("CPU Cores     :", os.cpu_count())
print("CPU Temp      :", round(temp, 1), "C")
print("Total RAM     :", round(mem['MemTotal'], 1), "GB")
print("Free RAM      :", round(mem['MemAvailable'], 1), "GB")
```

**OUTPUT:**

```
Hostname      : raspberrypi
Board Model   : Raspberry Pi 4 Model B Rev 1.4
Operating Sys : Linux 6.1.21-v8+ (aarch64)
Python Version: 3.11.2
CPU Cores     : 4
CPU  Temp   :  47.2  C
Total  RAM   :  3.7  GB
Free RAM     : 2.9 GB
```

**RESULT:**

Thus, the working of the Raspberry Pi has been studied and the board and system details have been displayed successfully using Python.

## Ex. No: 2 CONTROLLING LED USING ARDUINO UNO

**Date:16.07.2026**

**AIM:**

To control three LEDs using an Arduino Uno in the Wokwi simulator and make them glow one after another in sequence.

**PROCEDURE:**

Step 1: Open the Wokwi simulator (wokwi.com) and start a new Arduino Uno project.

Step 2: Add three LEDs and connect their anodes to digital pins 13, 12 and 11 of the Arduino Uno, and their cathodes to GND.

Step 3: In sketch.ino, configure pins 13, 12 and 11 as OUTPUT inside setup().

Step 4: Inside loop(), turn each LED ON for one second and then OFF, one after the other.

Step 5: Start the simulation and observe the LEDs glowing in sequence.

**CODE:**

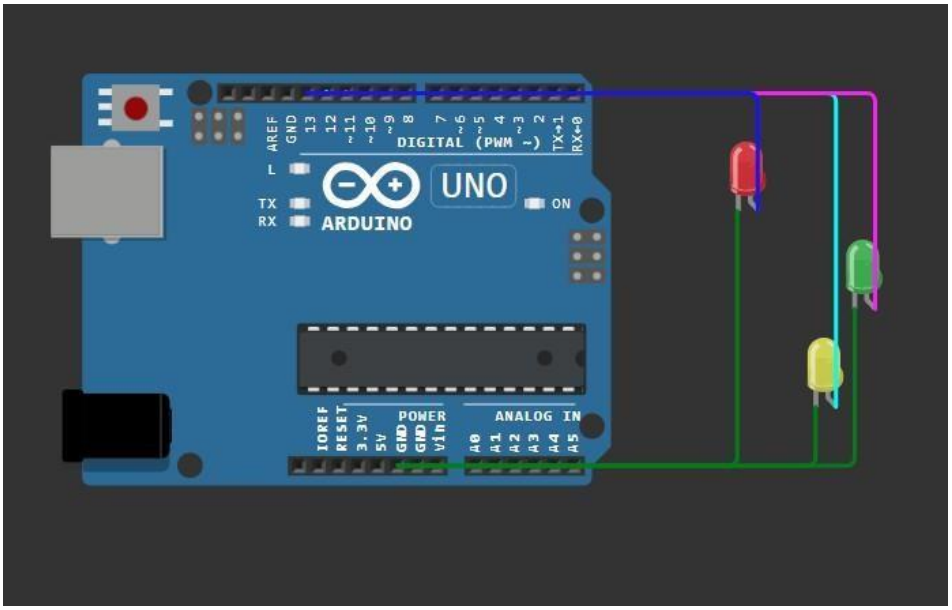
```
void setup()
{
  pinMode(13, OUTPUT);
  pinMode(12, OUTPUT);
  pinMode(11, OUTPUT);
}

void loop()
{
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);

  digitalWrite(12, HIGH);
  delay(1000);
  digitalWrite(12, LOW);

  digitalWrite(11, HIGH);
  delay(1000);
  digitalWrite(11, LOW);
}
```

**OUTPUT:**




**RESULT:**

Thus, the LEDs have been controlled using the Arduino Uno and made to glow in sequence successfully.

## Ex. No: 3 TRAFFIC SIGNAL USING LED LIGHTS WITH ESP32

**Date:23.07.2026**

**AIM:**

To implement a traffic signal using three LEDs and an ESP32 in the Wokwi simulator.

**PROCEDURE:**

Step 1: Open the Wokwi simulator (wokwi.com) and start a new ESP32 project.

Step 2: Add three LEDs (red, yellow and green) and connect them to GPIO pins 25, 26 and 27 of the ESP32, with their cathodes connected to GND.1.

Step 3: In sketch.ino, configure pins 25, 26 and 27 as OUTPUT inside setup().

Step 4: Inside loop(), switch ON one LED at a time: pin 25 for 4 seconds, pin 26 for 3 seconds and pin 27 for 5 seconds, keeping the other two OFF.

Step 5: Start the simulation and observe the traffic signal sequence.

**CODE:**

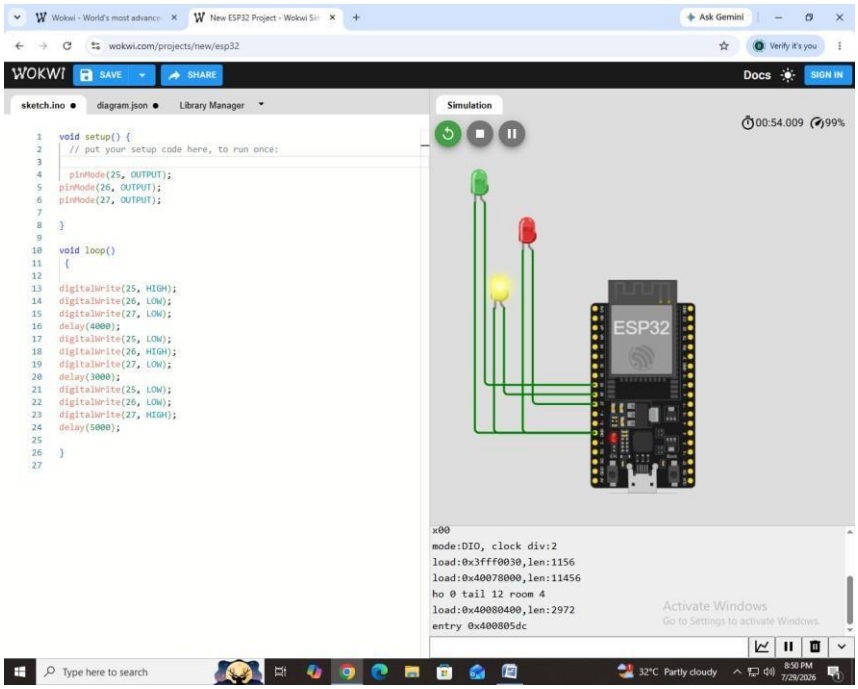
```
void setup() {
  pinMode(25, OUTPUT);
  pinMode(26, OUTPUT);
  pinMode(27, OUTPUT);
}

void loop() {
  digitalWrite(25, HIGH);
  digitalWrite(26, LOW);
  digitalWrite(27, LOW);
  delay(4000);

  digitalWrite(25,  LOW);
  digitalWrite(26, HIGH);
  digitalWrite(27,  LOW);
  delay(3000);

  digitalWrite(25,  LOW);
  digitalWrite(26,  LOW);
  digitalWrite(27, HIGH);
  delay(5000);
}
```

**OUTPUT:**




```
void setup() {
  // put your setup code here, to run once:

  pinMode(25, OUTPUT);
pinMode(26, OUTPUT);
pinMode(27, OUTPUT);

}

void loop()
 {

digitalWrite(25, HIGH);
digitalWrite(26, LOW);
digitalWrite(27, LOW);
delay(4000);
digitalWrite(25, LOW);
digitalWrite(26, HIGH);
digitalWrite(27, LOW);
delay(3000);
digitalWrite(25, LOW);
digitalWrite(26, LOW);
digitalWrite(27, HIGH);
delay(5000);

}
```

**RESULT:**

Thus, the traffic signal has been implemented using LEDs and the ESP32 successfully.

## Ex. No: 4 DESCRIBING GATEWAY AS A SERVICE DEPLOYMENT IN IOT TOOLKIT

**Date:23.07.2026**

**AIM:**

To deploy a gateway as a service on the Raspberry Pi that receives data from IoT devices, stores it and makes it available to applications through a REST interface.

**PROCEDURE:**

Step 1: Understand the gateway role: it connects the sensor devices to the network or cloud, collects the data and exposes it as a service.

Step 2: Install the toolkit packages on the Raspberry Pi using: pip install flask requests.

Step 3: Create gateway.py, a Flask service with a POST /data route to receive readings and GET /data and GET /status routes to serve them. Start it on port 5000.

Step 4: Create device.py, a simulated IoT device that sends temperature and humidity readings to the gateway.

Step 5: Run the gateway, then the device, and verify the stored data through the browser at http://<pi-ip>:5000/data.

**CODE:**

```
# ---------- gateway.py ----------
from flask import Flask, request, jsonify
from datetime import datetime

app = Flask(_name_)
readings = []

@app.route('/data', methods=['POST'])
def receive():
    d = request.get_json()
    d['time'] = datetime.now().strftime('%H:%M:%S')
    readings.append(d)
    print("Gateway received:", d)
    return jsonify(status='stored', count=len(readings))

@app.route('/data', methods=['GET'])
def send():
    return jsonify(readings)

@app.route('/status', methods=['GET'])
def status():
    return jsonify(service='IoT Gateway', running=True, total=len(readings))
```

```
app.run(host='0.0.0.0', port=5000)

# ---------- device.py ----------
import requests, random, time

URL = "http://localhost:5000/data"
for i in range(5):
    payload = {"device": "sensor-01",
               "temperature": round(random.uniform(25, 35), 1),
               "humidity": random.randint(50, 70)}
    r = requests.post(URL, json=payload)
    print("Sent:", payload, "->", r.json())
    time.sleep(2)
```

**OUTPUT:**

```
--- Terminal 1 (gateway.py) ---
 * Serving Flask app 'gateway'
 * Running on http://0.0.0.0:5000
Gateway received: {'device': 'sensor-01', 'temperature': 28.4, 'humidity': 61, 'time': '10:15:02'}
Gateway received: {'device': 'sensor-01', 'temperature': 31.2, 'humidity': 57, 'time': '10:15:04'}
Gateway received: {'device': 'sensor-01', 'temperature': 26.9, 'humidity': 66, 'time': '10:15:06'}

--- Terminal 2 (device.py) ---
Sent: {'device': 'sensor-01', 'temperature': 28.4, 'humidity': 61} -> {'count': 1, 'status': 'stored'}
Sent: {'device': 'sensor-01', 'temperature': 31.2, 'humidity': 57} -> {'count': 2, 'status': 'stored'}
Sent: {'device': 'sensor-01', 'temperature': 26.9, 'humidity': 66} -> {'count': 3, 'status': 'stored'}
```

**RESULT:**

Thus, the gateway has been deployed as a service and the device data has been collected and served successfully.

## Ex. No: 5 WEATHER MONITORING SYSTEM

**Date:30.07.2026**

**AIM:**

To implement a Weather Reporting System using ESP32 with a DHT22 temperature and humidity sensor.

**PROCEDURE:**

Step 1: Initialize the ESP32 and configure the DHT22 sensor.

Step 2: Read the temperature and humidity values from the sensor.

Step 3: Check the validity of the sensor data.

Step 4: Display the readings on the Serial Monitor.

Step 5: Repeat periodically (every few seconds).

**CODE:**

```
// Include Library: DHT sensor library for ESPx
#include "DHTesp.h"

#define DHT_PIN 15  // DHT22 data pin

DHTesp dht;

void setup() {
  Serial.begin(115200);
  dht.setup(DHT_PIN, DHTesp::DHT22);
}

void loop() {
  TempAndHumidity data = dht.getTempAndHumidity();

  Serial.print("Temperature:  ");
  Serial.print(data.temperature);
  Serial.println(" °C");

  Serial.print("Humidity:  ");
  Serial.print(data.humidity);
  Serial.println(" %");

  if (isnan(data.temperature) || isnan(data.humidity)) {
    Serial.println("Failed to read from DHT sensor!");
  }
```

```
  Serial.println("----------------------------");
  delay(3000);   // update every 3 seconds
}
```

**CIRCUIT CONNECTION:**

| DHT22 Pin | ESP32 Connection |
|---|---|
| VCC | 3.3V |
| GND | GND |
| DATA | GPIO 15 (DHT_PIN) |
| Pull-up resistor | 10kΩ between DATA and 3.3V |

**DEVICE SETUP:**




**OUTPUT:**

```
Temperature: 28.5 °C
Humidity: 66.3 %
----------------------------
Temperature: 29.1 °C
Humidity: 65.8 %
----------------------------
```

**RESULT:**

Thus, the program has been executed and the Weather Reporting System using ESP32 and the DHT22 sensor has been verified successfully.

## Ex. No: 6 IOT BASED SOIL MOISTURE MONITORING DEVICE

**Date:30.07.2026**

**AIM:**

To monitor the moisture level of the soil using a soil moisture sensor and an Arduino Uno in the Wokwi simulator, and to indicate dry soil using an LED.

**PROCEDURE:**

Step 1: Open the Wokwi simulator and start a new Arduino Uno project.

Step 2: Connect the sensor (a potentiometer is used to simulate the soil moisture sensor) with VCC to 5V, GND to GND and the signal pin to analog pin A0. Connect an LED with a resistor to digital pin 13.

Step 3: In sketch.ino, read the sensor value using analogRead() and print it on the Serial Monitor.

Step 4: If the value is below 400, the soil is Dry and the LED is switched ON; otherwise the soil is Wet and the LED is switched OFF.

Step 5: Start the simulation, vary the potentiometer and observe the messages on the Serial Monitor.

**CODE:**

```
const int soilPin = A0;
const int ledPin = 13;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  int soilValue = analogRead(soilPin);

  Serial.print("Soil Moisture Value: ");
  Serial.println(soilValue);

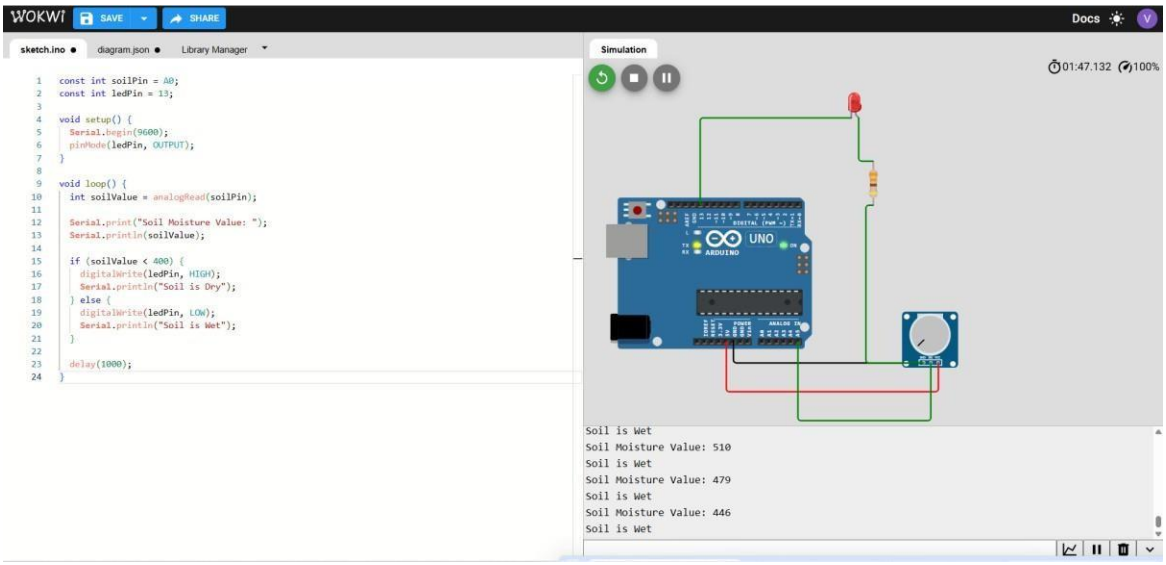
  if (soilValue < 400) {
    digitalWrite(ledPin, HIGH);
    Serial.println("Soil is Dry");
  } else {
    digitalWrite(ledPin, LOW);
    Serial.println("Soil is Wet");
```

```
  }

  delay(1000);
}
```

**OUTPUT:**

Soil Moisture Value: 510
Soil is Wet
Soil Moisture Value: 479
Soil is Wet
Soil Moisture Value: 446
Soil is Wet




**RESULT:**

Thus, the IoT based soil moisture monitoring device has been implemented and the soil condition has been detected successfully.

## Ex. No: 7 INSTALLING OPENCV AND DISPLAYING IMAGES

**Date:06.08.2026**

**AIM:**

To install the OpenCV library in Python and display an image in a window.

**PROCEDURE:**

Step 1: Open the terminal or command prompt and install OpenCV using: pip install opencv-python.

Step 2: Import the cv2 module and print its version to confirm the installation.

Step 3: Read the image file using cv2.imread().

Step 4: Display the image in a window using cv2.imshow().

Step 5: Wait for a key press using cv2.waitKey(0) and close all windows using cv2.destroyAllWindows().

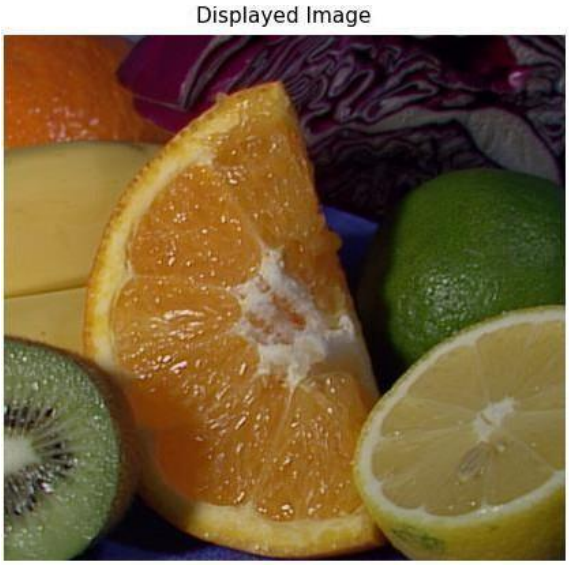
**CODE:**

```
# Install (terminal): pip install opencv-python
import cv2

print("OpenCV version:", cv2.__version__)

image = cv2.imread('fruits.jpg')
cv2.imshow('Displayed Image', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

**OUTPUT:**




**RESULT:**

Thus, OpenCV has been installed and an image has been displayed successfully.

## Ex. No: 8 READING AND WRITING IMAGES USING OPENCV

**Date:20.08.2026**

**AIM:**

To read an image, display its properties and write (save) a new image using OpenCV.

**PROCEDURE:**

Step 1: Read the colour image using cv2.imread() and display its shape, size and data type.

Step 2: Read the same image in grayscale mode using the cv2.IMREAD_GRAYSCALE flag.

Step 3: Save the grayscale image to disk using cv2.imwrite().

Step 4: Display the original and the saved images using cv2.imshow().

Step 5: Verify that the new file has been created in the working folder.

**CODE:**

```
import cv2

image = cv2.imread('butterfly.jpg')
print("Image Shape :", image.shape)
print("Image Size  :", image.size)
print("Data Type   :", image.dtype)

gray = cv2.imread('butterfly.jpg', cv2.IMREAD_GRAYSCALE)
cv2.imwrite('output_gray.jpg', gray)
print("Image saved as output_gray.jpg")

cv2.imshow('Original Image', image)
cv2.imshow('Saved Grayscale Image', gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

**OUTPUT:**

Image Shape : (356, 493, 3)
Image Size : 526524
Data Type : uint8
Image saved as output_gray.jpg




**RESULT:**

Thus, an image has been read, its properties displayed and a new image written using OpenCV.

## Ex. No: 9 DRAWING A RECTANGLE AND A CIRCLE

**Date:20.08.2026**

**AIM:**

To draw geometric shapes such as a rectangle, a circle and a line on an image using OpenCV.

**PROCEDURE:**

Step 1: Read the image using cv2.imread().

Step 2: Draw a rectangle using cv2.rectangle() by giving the top-left corner, bottom-right corner, colour and thickness.

Step 3: Draw a circle using cv2.circle() by giving the centre, radius, colour and thickness. A thickness of -1 fills the shape.

Step 4: Draw a line using cv2.line().

Step 5: Display the image with the drawn shapes.

**CODE:**

```
import cv2

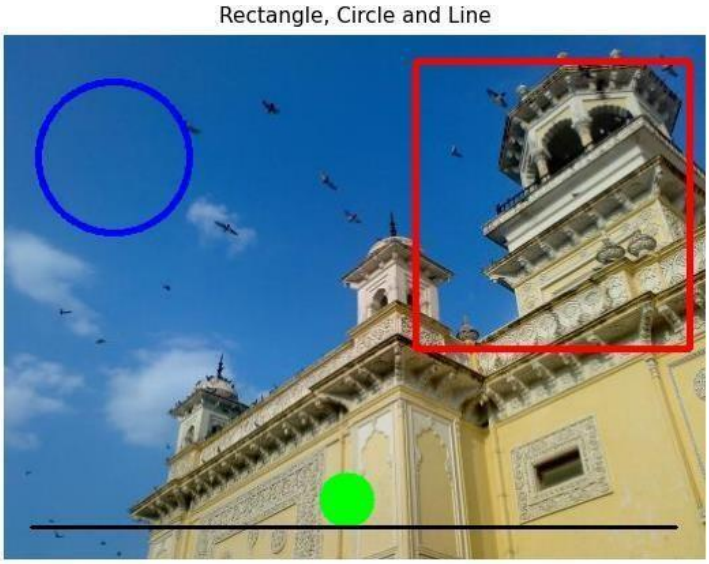
image = cv2.imread('home.jpg')

cv2.rectangle(image, (300, 20), (500, 230), (0, 0, 255), 3)
cv2.circle(image, (80, 90), 55, (255, 0, 0), 3)
cv2.circle(image, (250, 340), 20, (0, 255, 0), -1)
cv2.line(image, (20, 360), (490, 360), (0, 0, 0), 2)

cv2.imshow('Rectangle, Circle and Line', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

**OUTPUT:**

Rectangle, filled circle and line drawn on the image.




**RESULT:**

Thus, a rectangle and a circle have been drawn on an image using OpenCV.

## Ex. No: 10 TEXT IN IMAGES

**Date:27.08.2026**

**AIM:**

To write text on an image using different fonts, sizes and colours in OpenCV.

**PROCEDURE:**

Step 1: Read the image using cv2.imread().

Step 2: Choose a font such as cv2.FONT_HERSHEY_SIMPLEX, cv2.FONT_HERSHEY_COMPLEX or cv2.FONT_HERSHEY_SCRIPT_COMPLEX.

Step 3: Write the text using cv2.putText() by giving the text, position, font, scale, colour and thickness.

Step 4: Repeat with other fonts and colours at different positions.

Step 5: Display the image with the text.

**CODE:**

```
import cv2

image = cv2.imread('building.jpg')

cv2.putText(image, "Hello OpenCV", (30, 60),
        cv2.FONT_HERSHEY_SIMPLEX, 1.6, (0, 0, 255), 3)
cv2.putText(image, "IoT Lab Practical", (30, 570),
        cv2.FONT_HERSHEY_COMPLEX, 1.4, (0, 0, 0), 2)
cv2.putText(image, "Italic Font", (500, 300),
        cv2.FONT_HERSHEY_SCRIPT_COMPLEX, 1.8, (255, 0, 0), 2)

cv2.imshow('Text on Image', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

**OUTPUT:**

Text written on the image using putText().




**RESULT:**

Thus, text has been written on an image using OpenCV.

## Ex. No: 11 COLOR SPACE AND THRESHOLDING IN OPENCV

**Date:03.09.2026**

**AIM:**

To convert an image between colour spaces (BGR, Gray, HSV) and to apply thresholding techniques in OpenCV.

**PROCEDURE:**

Step 1: Read the image (OpenCV loads it in BGR format).

Step 2: Convert the image to grayscale and HSV using cv2.cvtColor().

Step 3: Apply simple thresholding (binary and binary inverse) with a threshold value of 127 using cv2.threshold().

Step 4: Apply Otsu's method to find the threshold value automatically.

Step 5: Display all the resulting images for comparison.

**CODE:**

```
import cv2

image = cv2.imread('stuff.jpg')

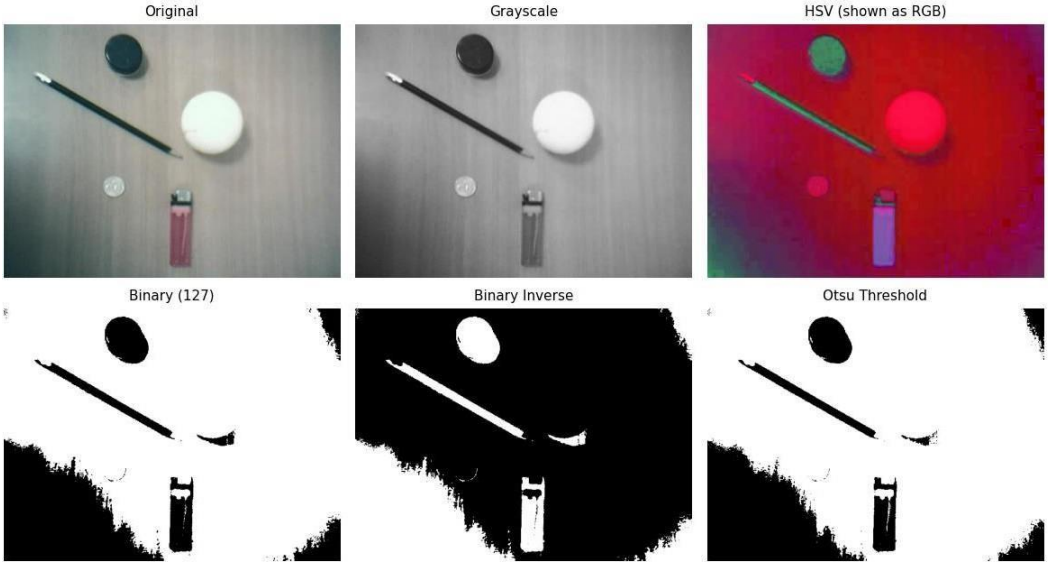
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
_, inverse = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY_INV)
otsu_val, otsu = cv2.threshold(gray, 0, 255,
                               cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print("Otsu's optimal threshold value:", otsu_val)

cv2.imshow('Original', image)
cv2.imshow('Grayscale', gray)
cv2.imshow('HSV', hsv)
cv2.imshow('Binary', binary)
cv2.imshow('Binary Inverse', inverse)
cv2.imshow('Otsu Threshold', otsu)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

**OUTPUT:**

Otsu's optimal threshold value: 124.0




**RESULT:**

Thus, colour space conversion and thresholding have been performed on an image using OpenCV.

## Ex. No: 12 FINDING CONTOURS

**Date:10.09.2026**

**AIM:**

To find and draw the contours (object boundaries) in an image using OpenCV.

**PROCEDURE:**

Step 1: Read the image and convert it to grayscale.

Step 2: Apply Gaussian blur to remove noise.

Step 3: Apply inverse Otsu thresholding so that the objects appear white on a black background, and ignore very small contours (noise).

Step 4: Find the contours using cv2.findContours() and print their count.

Step 5: Draw the contours on the image using cv2.drawContours() and display it.

**CODE:**

```
import cv2

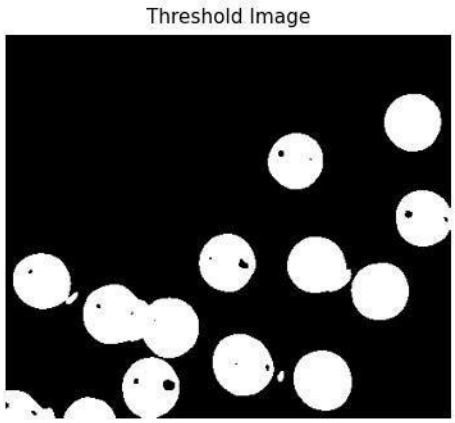
image = cv2.imread('smarties.png')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)

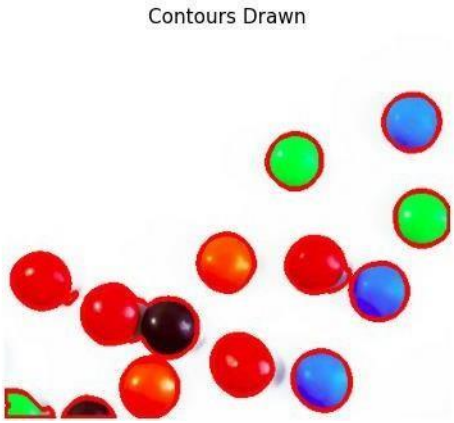
_, thresh = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
contours, hierarchy = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
                                       cv2.CHAIN_APPROX_SIMPLE)
# ignore very small contours (noise)
contours = [c for c in contours if cv2.contourArea(c) > 200]
print("Number of contours found:", len(contours))

cv2.drawContours(image, contours, -1, (0, 0, 255), 3)
cv2.imshow('Threshold Image', thresh)
cv2.imshow('Contours Drawn', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

**OUTPUT:**

Number of contours found: 13




**RESULT:**

Thus, the contours in an image have been found and drawn using OpenCV.

## Ex. No: 13 IMAGE EDGE DETECTION USING OPENCV

**Date:17.09.2026**

**AIM:**

To detect the edges in an image using the Canny, Sobel and Laplacian operators in OpenCV.

**PROCEDURE:**

Step 1: Read the image and convert it to grayscale.

Step 2: Apply the Canny edge detector using cv2.Canny() with lower and upper thresholds.

Step 3: Apply the Sobel operator in the X and Y directions using cv2.Sobel() and combine them.

Step 4: Apply the Laplacian operator using cv2.Laplacian().

Step 5: Display and compare the edge images.

**CODE:**

```
import cv2

image = cv2.imread('board.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

canny = cv2.Canny(gray, 100, 200)

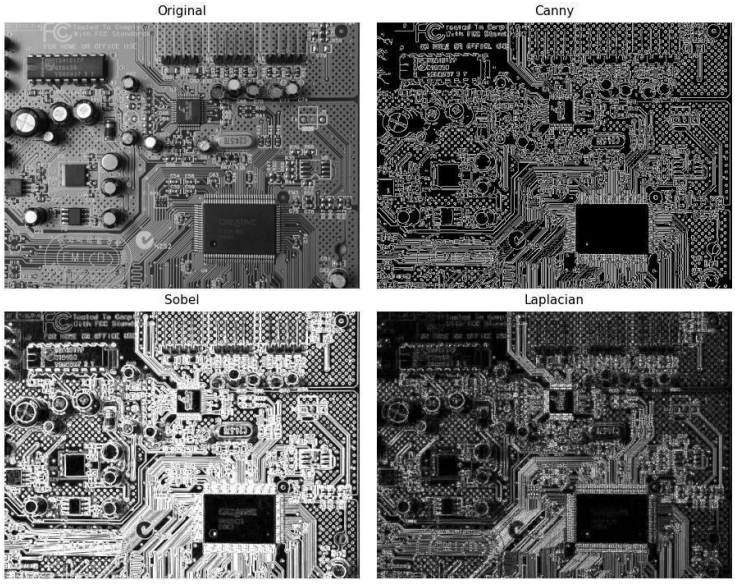
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.convertScaleAbs(cv2.magnitude(sobel_x, sobel_y))

laplacian = cv2.convertScaleAbs(cv2.Laplacian(gray, cv2.CV_64F))

cv2.imshow('Original', gray)
cv2.imshow('Canny', canny)
cv2.imshow('Sobel', sobel)
cv2.imshow('Laplacian', laplacian)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

**OUTPUT:**

Edges detected using Canny, Sobel and Laplacian operators.




**RESULT:**

Thus, the edges in an image have been detected using OpenCV.

## Ex. No: 14 IMAGE SCALING AND ROTATION USING OPENCV

**Date:24.09.2026**

**AIM:**

To resize (scale) and rotate an image using OpenCV.

**PROCEDURE:**

Step 1: Read the image and print its original width and height.

Step 2: Scale the image down by half and up by 1.5 times using cv2.resize().

Step 3: Create a rotation matrix for 45 degrees about the image centre using cv2.getRotationMatrix2D().

Step 4: Apply the rotation using cv2.warpAffine().

Step 5: Display the original, scaled and rotated images.

**CODE:**

```
import cv2

image = cv2.imread('baboon.jpg')
h, w = image.shape[:2]
print("Original Size :", w, "x", h)

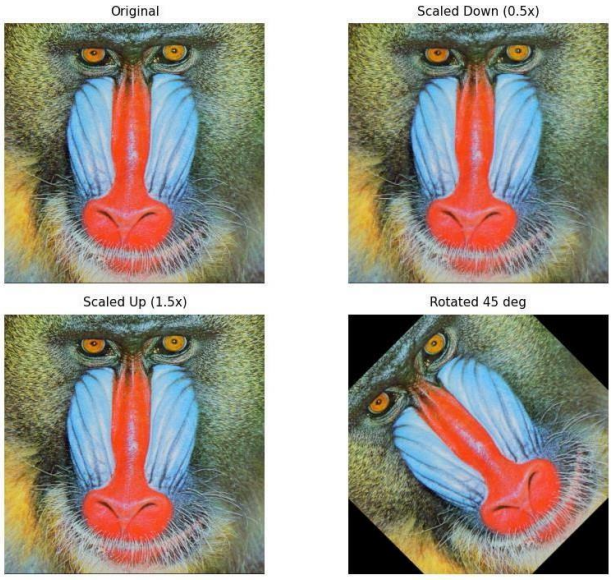
small = cv2.resize(image, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
big = cv2.resize(image, None, fx=1.5, fy=1.5, interpolation=cv2.INTER_CUBIC)
print("Scaled Down  :", small.shape[1], "x", small.shape[0])
print("Scaled Up    :", big.shape[1], "x", big.shape[0])

M = cv2.getRotationMatrix2D((w // 2, h // 2), 45, 1.0)
rotated = cv2.warpAffine(image, M, (w, h))
print("Rotated by 45 degrees about the image centre")

cv2.imshow('Original', image)
cv2.imshow('Scaled Down', small)
cv2.imshow('Scaled Up', big)
cv2.imshow('Rotated', rotated)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

**OUTPUT:**

Original Size : 512 x 512
Scaled Down : 256 x 256
Scaled Up : 768 x 768
Rotated by 45 degrees about the image centre




**RESULT:**

Thus, an image has been scaled and rotated using OpenCV.

## Ex. No: 15 IMAGE TRANSLATION AND IMAGE FILTERING USING OPENCV

**Date:01.10.2026**

**AIM:**

To translate (shift) an image and to smooth it using averaging, Gaussian, median and bilateral filters in OpenCV.

**PROCEDURE:**

Step 1: Read the image and get its width and height.

Step 2: Create a translation matrix that shifts the image 80 pixels right and 50 pixels down, and apply it using cv2.warpAffine().

Step 3: Apply averaging blur using cv2.blur() and Gaussian blur using cv2.GaussianBlur().

Step 4: Apply median blur using cv2.medianBlur() and bilateral filter using cv2.bilateralFilter().

Step 5: Display all the images and compare the results.

**CODE:**

```
import cv2
import numpy as np

image = cv2.imread('apple.jpg')
h, w = image.shape[:2]

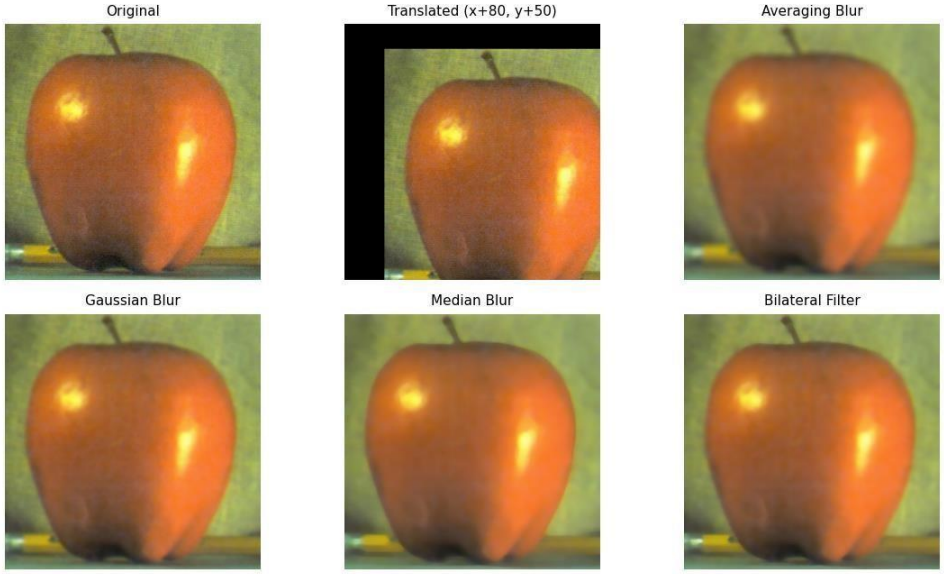
# Translation: shift 80 px right and 50 px down
T = np.float32([[1, 0, 80], [0, 1, 50]])
shifted = cv2.warpAffine(image, T, (w, h))

# Filtering
average = cv2.blur(image, (15, 15))
gaussian = cv2.GaussianBlur(image, (15, 15), 0)
median = cv2.medianBlur(image, 15)
bilateral = cv2.bilateralFilter(image, 15, 75, 75)

cv2.imshow('Original', image)
cv2.imshow('Translated', shifted)
cv2.imshow('Averaging Blur', average)
cv2.imshow('Gaussian Blur', gaussian)
cv2.imshow('Median Blur', median)
cv2.imshow('Bilateral Filter', bilateral)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

**OUTPUT:**

Image translated by (80, 50) pixels and filtered with averaging, Gaussian, median and bilateral filters.




**RESULT:**

Thus, an image has been translated and filtered using OpenCV.